

A Mathematical Investigation: Optimizing Evacuation Routes

Path Metrics and D* Lite Algorithm

August 28, 2026

Contents

1	Introduction	2
2	Step 1: Calculating Spherical Distance (Haversine)	2
3	Step 2: Accounting for Topography (Pythagorean Theorem)	2
4	Step 3: Calculating Kinematic Penalties	3
4.1	Gradient (Slope Penalty)	3
4.2	Sinuosity (Curvature Penalty)	3
5	Step 4: True Travel Time	4
6	The D* Lite Algorithm	4
6.1	The Limitation of A* (A-Star)	4
6.2	The Mathematics of D* Lite	4

1 Introduction

In the event of a natural disaster, finding the fastest evacuation route is a critical real-world problem. To a computer, a map is modeled as a graph containing nodes (intersections) and edges (roads). In order to determine the fastest route, we must mathematically calculate the "cost" (travel time) of every edge. This investigation details the mathematical sequence required to convert raw GPS data into physical travel times, and how the D* Lite algorithm utilizes this data to find an optimal path.

2 Terminologies and Abbreviations

To ensure a clear understanding of the mathematical models and algorithms discussed in this investigation, the following key terms and variables are defined:

- **Node / Vertex:** A distinct point on a map, such as an intersection, a town, or a GPS coordinate.
- **Edge:** The connection or path (e.g., a road) between two nodes.
- **Cost (Edge Cost):** The mathematical "weight" of an edge. In this study, the cost specifically refers to the *travel time* required to cross an edge.
- **GPX (GPS Exchange Format):** A data format containing recorded GPS tracks, including latitude (ϕ), longitude (λ), and elevation (z).
- **Haversine Formula:** An equation crucial in navigation, giving great-circle distances between two points on a sphere from their longitudes and latitudes.
- **Gradient (Slope):** The physical steepness of a road, calculated as the ratio of vertical elevation change (rise) to horizontal distance (run).
- **Sinuosity Index:** A mathematical ratio used to quantify how much a path curves or deviates from the shortest possible straight-line path.
- **Heuristic:** An informed mathematical guess or approximation (used by algorithms like A* and D* Lite) to estimate the remaining distance to the goal.
- V_{base} : The theoretical maximum base speed of a vehicle on a perfectly flat, straight road.

- V_{adj} : The final adjusted speed of a vehicle after mathematically applying penalties for steep slopes and sharp curves.
- $g(s)$: The calculated true cost (time) to travel from a specific node s to the goal.
- $rhs(s)$: The one-step lookahead cost, used in D* Lite to detect if a hazard has suddenly changed the travel time of a nearby road.

3 Step 1: Calculating Spherical Distance (Haversine)

The Earth is a sphere, meaning standard Euclidean geometry (like measuring a straight line on a flat piece of paper) will result in inaccurate distances over large areas. To accurately measure the ground distance between two GPS coordinates, we use the **Haversine Formula**, which calculates the "great-circle" distance on a sphere.

Given two points with latitudes ϕ_1, ϕ_2 and longitudes λ_1, λ_2 (in radians):

$$\Delta\phi = \phi_2 - \phi_1 \quad (1)$$

$$\Delta\lambda = \lambda_2 - \lambda_1 \quad (2)$$

We calculate the intermediate value a :

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (3)$$

We then calculate the angular distance c :

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \quad (4)$$

Finally, we multiply by the Earth's radius R (6,371,000 meters) to find the 2D ground distance d_{2d} :

$$d_{2d} = R \cdot c \quad (5)$$

4 Step 2: Accounting for Topography (Pythagorean Theorem)

The Haversine formula provides the distance as if we drew a straight line through the base of a hill. However, a vehicle must travel *over* the hill.

To calculate the actual physical road length, we apply the **Pythagorean Theorem**.

If the 2D ground distance d_{2d} forms the base of a right-angled triangle, and the elevation change $\Delta z = (z_2 - z_1)$ forms the height, the road acts as the hypotenuse:

$$d_{3d} = \sqrt{d_{2d}^2 + \Delta z^2} \quad (6)$$

5 Step 3: Calculating Kinematic Penalties

A vehicle can only maintain its maximum base speed (V_{base}) on a road that is perfectly flat and perfectly straight. We must mathematically penalize (reduce) the vehicle's speed based on the terrain.

5.1 Gradient (Slope Penalty)

The gradient measures the steepness of the road, defined mathematically as the ratio of "rise over run".

$$\text{Gradient} = \left(\frac{\Delta z}{d_{2d}} \right) \times 100\% \quad (7)$$

We apply a linear penalty P_{slope} , dictating that for every 1% of absolute gradient, the vehicle's speed is reduced by 2%. We cap this penalty at 80% (0.8) to ensure the vehicle retains a minimum forward velocity on extremely steep hills.

$$P_{slope} = \min(|\text{Gradient}| \times 0.02, 0.8) \quad (8)$$

5.2 Sinuosity (Curvature Penalty)

Sinuosity measures how "winding" a road is by comparing the actual path length to the straight-line distance. A perfectly straight road has a ratio of 1.0. A zig-zagging mountain road might have a ratio of 1.5 (meaning the actual road is 50% longer than a straight line).

$$\text{Sinuosity} = \frac{\sum d_{2d}}{\text{Straight-Line Distance}} \quad (9)$$

We introduce a curvature penalty P_{curve} that reduces speed by 5% for every 0.1 increment above 1.0, capped at 50% (0.5).

$$P_{curve} = \min(\max(\text{Sinuosity} - 1.0, 0) \times 0.5, 0.5) \quad (10)$$

6 Step 4: True Travel Time

We determine the vehicle's adjusted speed (V_{adj}) by subtracting our calculated penalty modifiers from the theoretical base speed ($V_{base} = 8.33$ m/s, approx. 30 km/h).

$$V_{adj} = V_{base} \times (1 - P_{slope}) \times (1 - P_{curve}) \quad (11)$$

Finally, we apply the fundamental kinematic equation to determine the true travel time cost (in seconds) for that specific road segment:

$$\text{Time} = \frac{d_{3d}}{V_{adj}} \quad (12)$$

7 The D* Lite Algorithm

Once the mathematical time cost is calculated for every road, the system must navigate the vast network of nodes to find the optimal path.

7.1 The Limitation of A* (A-Star)

Standard GPS applications utilize the A* algorithm, which explores nodes by calculating a heuristic cost from the start to the goal. While highly efficient for static maps, A* fails in disaster scenarios. If a road is suddenly reported as flooded (dynamic edge cost change), A* is invalidated and must completely recalculate the entire path from scratch, which is computationally expensive and slow during an emergency.

7.2 The Mathematics of D* Lite

D* Lite (Dynamic A* Lite) is a specialized incremental search algorithm optimized for dynamic, unknown environments. It operates on two foundational principles that differentiate it from A*:

1. **Backward Search:** D* Lite searches in reverse, starting from the Goal (evacuation center) and working backwards to the Start (evacuee location). Mathematically, it calculates the true cost $g(s)$ to reach the goal from any given state s .
2. **Local Consistency (rhs values):** Alongside the actual cost $g(s)$, D* Lite maintains a one-step lookahead value called $rhs(s)$.

$$rhs(s) = \min_{s' \in Succ(s)} (c(s, s') + g(s')) \quad (13)$$

Where $c(s, s')$ is the travel time cost of the edge from state s to its successor s' .

Handling Dynamic Hazards:

A node is considered "consistent" when $g(s) = rhs(s)$. If a disaster strikes and a road's travel time (c) instantly increases due to flooding, the $rhs(s)$ value of the affected node changes, making it "inconsistent".

Instead of discarding all calculations like A*, D* Lite simply places the inconsistent node into a priority queue. It then cascades the mathematical updates only to the directly affected neighboring nodes, reusing the previously calculated mathematical costs for the rest of the map. This allows D* Lite to dynamically and instantaneously output a new optimal evacuation route the moment a hazard occurs.